《最优化算法与理论》实验报告

实验序号:	1
实验内容:	线性搜索法
班　　级:	信息 2301
学　　号:	202310040128
姓　　名:	雷龙浩
完成日期:	2026 年 03 月 21 日

目录

1 实验概述与分析 **3**
1.1 实验目标与问题描述 3
1.2 算法原理与实现逻辑描述 3
1.2.1 黄金分割法： 3
1.2.2 Armijo 型线搜索描述： 3
1.2.3 Wolfe-Powell 型线搜索描述： 3
1.3 算法程序实现 4

2 固定方向 d_0 的三种方法运行结果及参数敏感性分析 **6**
2.1 理论分析 6
2.2 实验结果统计 7
2.3 参数分析 7

3 最速下降法的三种方法运行结果及参数敏感性分析 **8**
3.1 最速下降法代码展示 8
3.2 参数分析 9

1 实验概述与分析

1.1 实验目标与问题描述

本实验旨在研究非线性优化中的线搜索（Line Search）策略。针对目标函数 $f(x) = (x_1 - x_2)^2 + x_2^2$，在给定点 x_0 和方向 d_0 下，对比三种不同性质的步长确定方法：

- 精确搜索：黄金分割法（求取使函数值最小的绝对最优步长）。
- 非精确搜索（充分下降）：Armijo 准则（保证函数值下降，计算开销小）。
- 非精确搜索（高效下降）：Wolfe-Powell 准则（兼顾下降量与梯度变化，保证算法稳定性）。

1.2 算法原理与实现逻辑描述

1.2.1 黄金分割法：

- 该算法通过**缩小区间**的策略 (类似于二分法) 寻找一元函数 $\phi(\alpha)$ 的极小点。
- 它利用黄金分割比例 $\tau \approx 0.618$ 在初始区间内取测试点。
- 逻辑核心：每次迭代比较两个测试点的函数值，剔除较大点以压缩搜索区间，直到区间长度小于设定的精度 ϵ。
- 特点：不依赖导数信息，但对初始区间的选择较为敏感。对于单峰函数可以稳定收敛到最优。

1.2.2 Armijo 型线搜索描述：

- 这是一种典型的回溯算法，旨在寻找一个满足“充分下降”条件的步长。
- 充分下降条件：$f(x_k + \alpha d_k) \leq f(x_k) + \sigma_1 \alpha \nabla f(x_k)^T d_k$。
- 实现逻辑：从一个初始大步长开始，如果不满足条件，则说明步长太大了，通过因子 ρ 不断衰减 α，直到步长落入由 σ_1 确定的斜率控制线之下。
- 缺点：Armijo 条件只能保证步长不过大，但不能保证步长不过小。

1.2.3 Wolfe-Powell 型线搜索描述：

- 在 Armijo 条件的基础上增加了曲率条件，防止步长过小导致迭代效率低下。
- 双重约束：既要满足函数值显著下降（充分下降条件），又要保证新点处的坡度变平缓（曲率条件）。
- 曲率条件：$\nabla f(x_k + \alpha_k d_k)^T d_k \geq \sigma_2 \nabla f(x_k)^T d_k$

- 实现逻辑：采用“回溯 + 外推”的组合手段。先找到满足下降条件的区间，再通过类似插值的方式在其中搜寻满足曲率要求的高质量步长。

我们用 Python 编写程序实现了黄金分割法（程序见作业附件压缩文件包）. 程序代码主体如下：

```python
# 精确线性搜索_黄金分割法，递归形式
# f(xk + alpha*dk) = phi(alpha)
# 计算 argmin phi 等价与计算最佳alpha
# 为了能在最速下降法中统一调用，加上了 xk, dk, f, grad_f 参数，并将初始区间设为默认参数 ak=0.0, bk=2.0
def golden_ratio_line_search(xk, dk, f, grad_f, ak=0.0, bk=2.0, counter=0, tol=1e-5):
    # 为了能在最速下降法中统一调用，在内部把多维问题降维，这样外部调用就不需要单独传 phi 了
    phi = lambda a: f(xk + a * dk)

    # goal: solve argmin phi(alpha) for alpha in [ak, bk]
    _lambda = 0.618
    counter += 1

    if abs(bk - ak) < tol:
        return (ak + bk) / 2 , counter
    # 计算区间内部的两个测试点 lam 和 mu
    # lam:= L mu:= R
    lam = bk - _lambda * (bk - ak)
    mu = ak + _lambda * (bk - ak)
    if phi(lam) < phi(mu):
        # 如果左侧内部点的值更小，说明极小点在左侧区间 [ak, mu] 内
        return golden_ratio_line_search(xk, dk, f, grad_f, ak, mu, counter=counter, tol=tol)
    else:
        # 如果右侧内部点的值更小(或相等)，说明极小点在右侧区间 [lam, bk] 内
        return golden_ratio_line_search(xk, dk, f, grad_f, lam, bk, counter=counter, tol=tol)

```

图 1: 黄金分割法程序.

1.3 算法程序实现

我们用 Python 编写程序实现了 Armijo 型线搜索法（程序见作业附件压缩文件包）. 程序代码主体如下：

```python
# Armijo 条件线搜索
def armijo_line_search(xk, dk, f, grad_f, alpha_init=1.0, sigma1=1e-4, rho=0.5):
    assert 0 < sigma1 < 0.5, "sigma1 must be in (0, 0.5)"
    assert 0 < rho < 1, "rho must be in (0, 1)"
    counter = 1
    alpha = alpha_init
    while f(xk + alpha * dk) > f(xk) + sigma1 * alpha * np.dot(grad_f(xk), dk):
        alpha *= rho
        counter += 1
    return alpha , counter
```

图 2: Armijo 型线搜索法.

我们用 Python 编写程序实现了 Wolfe-Powell 型线搜索（程序见作业附件压缩文件包）. 程序代码主体如下：

```python
# Wolfe-Powell 条件线搜索
def wolfe_powell_line_search(xk, dk, f, grad_f, alpha=1.0, beta=0.5, sigma1=1e-4, sigma2=0.9, rho=0.5):
    assert 0 < sigma1 < sigma2 < 1, "sigma1 and sigma2 must satisfy 0 < sigma1 < sigma2 < 1"
    assert 0 < beta
    def condition1(xk=xk, dk=dk, f=f, grad_f=grad_f, alpha=alpha, sigma1=sigma1):
        # 同 Armijo 条件线搜索条件
        return f(xk + alpha * dk) <= f(xk) + sigma1 * alpha * np.dot(grad_f(xk), dk)

    def condition2(xk=xk, dk=dk, f=f, grad_f=grad_f, alpha=alpha, sigma2=sigma2):
        # 曲率条件
        return np.dot(grad_f(xk + alpha * dk), dk) >= sigma2 * np.dot(grad_f(xk), dk)

    def iter1(xk=xk, dk=dk, f=f, grad_f=grad_f, beta=beta, rho=rho, counter=1):
        # 算法中第三步的迭代
        i = 1
        alpha = beta
        # 迭代调整
        while not condition1(xk, dk, f, grad_f, alpha=alpha):
            alpha = rho**i * beta
            i += 1
        return alpha, counter+i-1

    def iter2(alpha0, beta, xk=xk, dk=dk, f=f, grad_f=grad_f, rho=rho, counter=1):
        # 算法中第四步的迭代
        # alpha = alpha0 + rho^i * (beta - alpha0)
        i = 1
        # alpha = alpha0 + rho**0 * (beta - alpha0) = beta
        alpha = beta
        while not condition1(xk, dk, f, grad_f, alpha=alpha):
            # 迭代调整
            alpha = alpha0 + rho**i * (beta - alpha0)
            i += 1
        return alpha, counter+i-1

    counter = 1
    if condition1(alpha=alpha) and condition2(alpha=alpha):
        # alpha 默认值为1
        return alpha, counter  # 如果 alpha 满足条件，直接返回

    alpha, counter = iter1(counter=counter)  # step2:进行第一阶段的迭代
    while True:
        # step3:检查是否满足条件1和条件2
        if condition2(alpha=alpha):
            return alpha, counter  # 如果满足曲率条件，返回当前 alpha
        beta = alpha / rho  # 更新 beta
        # step4:进行第二阶段的迭代
        alpha, counter = iter2(alpha0=alpha, beta=beta, counter=counter)  # 进行第二阶段的迭代
```

图 3: Wolfe-Powell 型线搜索法.

2　固定方向 d_0 的三种方法运行结果及参数敏感性分析

2.1　理论分析

计算函数 $f(x)=(x_1-x_2)^2+x_2^2$ 在点 $x_0=(0,1)^T$ 处,沿着方向 $d_0=-(1,1)^T=(-1,-1)^T$ 的精确步长 α。

对于黄金分割法，我们的目标是构造一元函数 $\phi(\alpha)$ $\alpha>0$，使得 $\phi(\alpha)=f(x_0+\alpha d_0)$ 最小。

对于该题，确定位置向量：

$$x_0+\alpha d_0=\begin{pmatrix}0\\1\end{pmatrix}+\alpha\begin{pmatrix}-1\\-1\end{pmatrix}=\begin{pmatrix}-\alpha\\1-\alpha\end{pmatrix}$$

代入原函数 $f(x)$：

$$\begin{aligned}\phi(\alpha)&=f(-\alpha,1-\alpha)\\&=[-\alpha-(1-\alpha)]^2+(1-\alpha)^2\\&=(-1)^2+(1-\alpha)^2\\&=1+(1-\alpha)^2\end{aligned}$$

显然，这是一个关于 α 的二次函数，其极小值点在 $\alpha=1$ 处。故在黄金分割法中，我们可以选取区间迭代 $[0,2]$ 来逼近最优步长 $\alpha=1$ 。

2.2　实验结果统计

表 1: 固定起点 x0 和固定方向 d0 的线搜索求步长，参数敏感性分析

Method	Parameter	Alpha	Iters
GoldenRatio	$eps = 0.01$	1.000020	13
GoldenRatio	$eps = 0.0001$	1.000020	22
GoldenRatio	$eps = 1e-06$	1.000000	32
Armijo	$sigma1 = 0.0001$	1.250000	3
Armijo	$sigma1 = 0.1$	1.250000	3
Armijo	$sigma1 = 0.2$	1.250000	3
Armijo	$rho = 0.2$	1.000000	2
Armijo	$rho = 0.5$	1.250000	3
Armijo	$rho = 0.8$	1.638400	6
Wolfe	$sigma2 = 0.1$	1.000000	1
Wolfe	$sigma2 = 0.5$	0.500000	1
Wolfe	$sigma2 = 0.9$	0.500000	1
Wolfe	$beta = 0.1$	0.100000	1
Wolfe	$beta = 0.5$	0.500000	1
Wolfe	$beta = 2.0$	1.000000	2
Wolfe	$beta = 5.0$	1.250000	3

2.3　参数分析

1. 黄金分割法（精确搜索）
   数据表明，黄金分割法能够极其稳定地逼近理论最优解 $\alpha = 1.0$。随着容差 eps 从 0.01 缩小至 10^{-6}，迭代次数（Iters）从 13 次显著增加到 32 次。精确线搜索具有对数级别的收敛稳定性，不依赖于目标函数的导数信息。但在要求高精度（10^{-6}）时，大量的内部函数估值迭代会导致极大的计算开销。

2. Armijo 准则（回溯型非精确搜索）
   （注：从前面的分析可知最优步长为，$\alpha_{init} = 1.0$ 以下实验采用了较大的初始尝试步长，$\alpha_{init} = 5.0$ 以触发回溯）

   - **对 σ_1（充分下降系数）的敏感性较低：**当 σ_1 在 0.0001 到 0.2 之间变化时，最终求得的步长（1.25）和迭代次数（3 次）完全没有变化。这说明对于当前平滑的二次函数，只要初始步长进入了下降区域，绝大多数合理的 σ_1 都能轻易被满足，算法对该参数具有很强的鲁棒性。

- **对 ρ（步长衰减因子）的敏感性较高：**当 $\rho = 0.8$（衰减慢）时，步长在回溯中缩减得很精细，最终停在 $\alpha = 1.6384$（$5.0 \times 0.8^4 \approx 2.048$，再乘 0.8 为 1.6384）。虽然满足了下降条件，但这导致了最多的迭代次数（6 次），且步长距离最优解 1.0 较远。当 $\rho = 0.2$ 时 $5.0 \times 0.2 = 1.0$，恰好命中了极小值点。因此，ρ 决定了搜索网格的粗细。ρ 越大，搜索越细致但开销越大；ρ 越小，速度越快但容易错过优质解。工程上通常取 $\rho = 0.5$（二分搜索）作为折中。

3. **Wolfe-Powell 准则（双重条件搜索**

- **对 σ_2（曲率条件系数）的敏感性：**当 $\sigma_2 = 0.1$ 时（条件严苛，要求坡度极度平缓），算法输出了极其精确的最优解 $\alpha = 1.0$。当 $\sigma_2 = 0.5$ 或 0.9 时（条件宽松），算法接受了次优解 $\alpha = 0.5$。由此看出 σ_2 直接控制了非精确搜索逼近精确搜索的程度。σ_2 越小，求得的步长越精确。
- **对 β（回溯系数）的敏感性：**当 β 较小（$0.1, 0.5$）时，算法极易在 1 次迭代内满足条件，但得到的步长偏短（0.1）。当 β 较大（$2.0, 5.0$）时，算法有更大的空间通过回溯（乘以 ρ）寻找更长、更优质的步长（最终得到了 1.0 和 1.25 更接近 1），但内部迭代次数增加至 2-3 次。

3　最速下降法的三种方法运行结果及参数敏感性分析

3.1　最速下降法代码展示

```python

# 定义最速下降法
def steepest_descent(x0, line_search_func, tol=1e-5, max_outer_iter=1000, **ls_params):
    xk = np.array(x0, dtype=float)
    for i in range(max_outer_iter):
        gk = grad_f(xk)
        # 停止准则：梯度范数极小
        if np.linalg.norm(gk) < tol:
            break

        dk = -gk # 最速下降方向

        # 传入 xk, dk 让线搜索算法动态计算当前步长
        alpha, _ = line_search_func(xk, dk, f, grad_f, **ls_params)
        xk = xk + alpha * dk

    return xk, f(xk), i

```

图 4: 最速下降法.

表 2: 最速下降法的三种方法运行结果

Method	**Parameter**	**Opt Value** $f(x^*)$	**SD Iters**
SD + Golden	$\epsilon = 10^{-4}$	2.48×10^{-10}	7
SD + Armijo	$\rho = 0.2$	1.31×10^{-11}	15
SD + Armijo	$\rho = 0.5$	2.91×10^{-11}	33
SD + Armijo	$\rho = 0.8$	1.01×10^{-11}	42
SD + Wolfe-Powell	$\sigma_2 = 0.1$	2.91×10^{-11}	33
SD + Wolfe-Powell	$\sigma_2 = 0.5$	2.91×10^{-11}	33
SD + Wolfe-Powell	$\sigma_2 = 0.9$	2.91×10^{-11}	33

3.2 参数分析

1. **黄金分割法**当采用黄金分割法求解精确步长时，最速下降法仅需 7 次外层迭代即可收敛，远低于非精确搜索。其理论依据在于：精确搜索保证了相邻两次迭代的搜索方向严格正交（$\nabla f(x_{k+1})^T \nabla f(x_k) = 0$），在当前方向上取得了最大下降量。然而，这 7 步外层迭代的代价是每一次都需要在内部执行数十次的黄金分割区间收缩运算。

2. **Armijo 准则**实验数据表明，衰减因子 ρ 显著影响最速下降法的全局迭代次数（$\rho = 0.8$ 时为 42 次，$\rho = 0.2$ 时为 15 次）。其理论依据在于：对于存在狭长山谷特征的二次函数，较大的 ρ 导致回溯步长衰减缓慢、单步跨度偏大，算法极易越过谷底到达对侧，从而加剧了最速下降法固有的**锯齿现象**，使得外层迭代次数激增。相反，较小的 ρ 激进地缩小了搜索步长，在一定程度上抑制了越过谷底的震荡，改善了收敛轨迹。

3. **Wolfe-Powell 准则**实验表明，在不同曲率系数 $\sigma_2 \in \{0.1, 0.5, 0.9\}$ 下，算法的外层迭代次数恒为 33 次，且与 $\rho = 0.5$ 的 Armijo 准则表现完全一致。其理论依据在于：对于本实验中性质良好的平滑二次凸函数，算法在第一阶段（按默认 $\rho = 0.5$ 回溯）找到的满足充分下降条件的试探步长 α_k，其切线斜率已经足够平缓，自然满足了所有的曲率条件限制（包括最严苛的 $\sigma_2 = 0.1$）。因此，算法未触发第二阶段的扩展插值逻辑，直接退化为标准 Armijo 搜索。这证明 Wolfe 准则在良态区域内不产生额外计算开销，仅在劣态区域起到防止步长过短的保护作用。